

MACE: A Multi-core Agentic Co-design Engine for Autonomous OpenPiton Bring-Up

Rana Umar Nadeem

CHIA A3 Hackathon Submission, September 2026

Abstract—MACE is an agentic AI system that builds and verifies multicore hardware on its own, running on the CHIA framework. Phase 1 adds `chia_openpiton`, a CHIA adapter exposing OpenPiton’s `configure`, `build`, `simulate`, `run`, and `collect` operations, validated with 208 tests. Phase 2 adds the MACE loop: a planner breaks a hardware objective into tasks, agents run them in parallel, and a deterministic integrator gates every change against Verilator simulation before accepting it. A failure-analysis agent diagnoses failures and the planner replans around them.

We tested MACE against two baselines, manual mesh scaling and a one-shot LLM prompt, on all three supported cores, with every result derived from a hardware run or RTL simulation, not an agent’s self-report. We added PicoRV32 as a third core, resolving several RTL bugs that kept it from passing under Verilator. We also found and fixed three OpenPiton bugs that kept 2x2 and 4x4 meshes from passing; with them fixed, MACE’s loop passes both sizes for Ariane and PicoRV32. The one-shot baseline failed on all three cores in three different ways; MACE passed ariane and correctly stopped within its retry budget on the other two instead of reporting a false pass. The artifact pins its toolchain in a Nix flake, entered with a single `nix develop`.

1. Introduction

Current agentic coding systems are typically benchmarked on isolated tasks or toy problems [1], [2], which fails to capture the complexity of actual hardware construction. Bringing up a multicore system means navigating a large RTL codebase (OpenPiton [3] spans hundreds of thousands of lines across network-on-chip, cache-coherence, tile, and platform logic), a fragile toolchain (version skew, cross-compilation bugs, checkout-specific issues), and verification that costs minutes to hours per attempt rather than milliseconds.

MACE is designed for this high-friction environment. Given a hardware objective, such as “bring up a 2x2 mesh” or “verify workload W passes,” MACE plans, implements, builds, simulates, diagnoses failures against Verilator [4] output, and iterates. Every claim of success is gated by Verilator simulation; MACE never accepts an agent’s self-report.

1.1. Features

MACE contributes:

- `chia_openpiton` (Sec. 3): a CHIA adapter for OpenPiton with a full `configure/build/run/collect` interface, built the same way as CHIA’s `gem5` and `ESP` adapters, so it can be upstreamed directly.
- The MACE loop (Sec. 4): planning, parallel task execution, deterministic integration, and LLM-driven failure triage, tested end-to-end on hardware with cost, time, and task metrics.
- A third core, PicoRV32 (Sec. 5), brought up by fixing pre-existing bugs in its upstream OpenPiton integration, which OpenPiton’s continuous integration builds but never runs.
- An interactive CLI and two further capabilities (Sec. 6): Verilator code coverage as a second verification axis, and a second working LLM backend.

2. Background

CHIA [5] provides the agent, tool, and dispatch layer this project builds on: colocated resource-pinned worker nodes, an LLM abstraction layer spanning multiple backends, and Ray-based [6] remote dispatch and caching. OpenPiton [3] is Princeton’s open-source, silicon-proven manycore research framework. Tiles connect through a 2D-mesh network-on-chip and speak a custom MESI-coherence protocol (L15, descended from OpenSPARC T1’s PCX/CPX bus) to reach the NoC. OpenPiton supports three tile cores, the latter two added through its BYOC framework [7]: OpenSPARC T1 (`sparc`), Ariane/CVA6 [8] (`ariane`, RV64GC, coherent), and PicoRV32 [9] (`pico`, RV32I, non-coherent). Before this project, CHIA had no OpenPiton adapter.

Related work. LLM-based hardware design has mostly been evaluated at module scale. VerilogEval [2] scores single-module Verilog generation, ChipNeMo [10] adapts LLMs to chip-design tasks, and AutoChip [11], RTL-Fixer [12], VerilogCoder [13], and MAGE [14] feed compiler or simulation results back to one or more agents to repair generated RTL. In software, SWE-bench [15] moved evaluation from isolated functions [1] to repository-level issues. MACE applies the same agent-with-feedback idea at system scale: rather than writing one module against a small testbench, it plans and verifies bring-up of an existing manycore, gated by whole-system Verilator simulation.

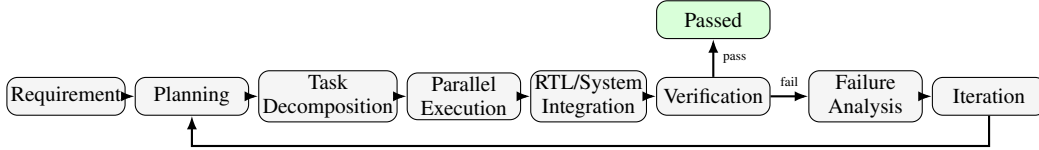


Figure 1. The MACE pipeline. Verification gates every integrated change against Verilator simulation rather than an agent’s self-report. The run ends when every gate workload passes; any failure triggers failure analysis and a replan rather than a bare retry.

3. Phase 1: The OpenPiton Adapter

Design. `OpenPitonWorkspaceNode` is a `CHIA ColocatedNode` exposing `configure`, `build`, `run`, `regress`, `collect`, and `clean` as resource-pinned remote functions. OpenPiton’s template preprocessor writes generated `.tmp.v` files into the source tree on every build, so the adapter gives each worker its own checkout, reserved through one `openpiton` resource unit, instead of locking a shared one. Pure parsers read build and run verdicts from OpenPiton’s transcripts (`sim.log`, `status.log`), using the strings its RTL monitors emit; a verdict never comes from an agent’s claim.

Environment hardening. The build surfaced four infrastructure bugs, fixed in one idempotent patch script: `binutils 2.38+` splitting `zicsr/zifencei` out of base RV64I, which breaks the boot ROM’s assembly; GCC 15’s C23 default breaking a legacy function call; broken git symlinks on a Windows-mounted checkout; and CRLF shebangs in 66 files. A Nix flake [16] pins the toolchain, including a Verilator release (5.052) that avoids two Verilator bugs this RTL triggers.

Build-result caching. `build()` skips the simulator when a matching successful-build marker exists: rebuilding an identical configuration dropped from 104s to 0.00s, so an iteration that only changes the test never rebuilds.

Results. 208 tests across three tiers (pure-unit, live-Ray with a stub simulator, and hardware) all pass. Hardware acceptance tests pass for a 1x1 Ariane build and run (`hello_world.c`, compiled and executed by the adapter itself) and for two independent checkouts building in parallel: 176s versus 324s serial. One acceptance test, 2x2 Ariane on a GCP worker, remains blocked by upstream infrastructure dependencies (Sec. 8).

4. Phase 2: The MACE Loop

Pipeline. Figure 1 shows the loop. A planning agent turns a `MaceSpec` (core, target mesh, gate workloads, objective, budget) into a task DAG, and workers dispatch ready tasks in parallel through Ray, each pinned to its own checkout. A deterministic integrator, with no LLM, applies accepted diffs in dependency order, then builds and runs every gate workload; each workload’s *verdict*, parsed from simulator output, must read `pass`. On any failure, a failure-analysis agent with read-only tools (`log grep`, file collection, a fixture diff, and an `objdump` symbol check) diagnoses the transcript, and the planner replans around its proposed fix.

Gate workloads surface an RTL bug. Writing the gate workloads (barrier-and-atomic-counter, producer/consumer,

hart-id scatter/gather) exposed a memory-visibility bug: a plain `volatile` read of a shared value was not reliably visible from the same hart shortly after the write, while an atomic fetch-and-add-zero read always was. Every gate workload now reads shared values through this `atomic_read()` helper.

Resource enforcement. MACE checks all three `Budget` dimensions (iterations, USD, wall-clock) before each iteration starts, and verifies gate-workload checksums before any LLM call or checkout access, so a task cannot tamper with its own grading. Every run in this paper together cost \$37 of a \$300 GCP credit allocation, covering simulation compute and LLM inference; development used a \$20/month Claude subscription, outside that budget.

Deterministic replay. MACE cache-tags LLM, build, and run decisions and re-serves them at no token or compute cost. Per CHIA’s cache implementation, this replays *decisions* (response text, pass/fail artifacts), not the source edits an agent made; those remain git’s job.

5. A Third Core: PicoRV32

Integrating arbitrary core RTL into OpenPiton means navigating fundamental architectural differences rather than wiring up a standard bus. OpenPiton has no reusable core-to-NoC bridge: under BYOC, each core reaches the L15 through its own transducer, and even Ariane’s AXI4 output goes unused, since a hand-written encoder taps CVA6’s internal pre-AXI cache-miss stream instead. The dividing line is whether the core has its own coherent cache, not whether it speaks a standard bus: a new core with one, as Ariane has, needs new coherence-adapter RTL, beyond this project’s timeline. PicoRV32 does not: a complete, TODO-free upstream transducer (`pico_ll15_transducer.v` and friends) already exists. OpenPiton’s CI builds it under Verilator but never runs it: the run job is commented out, targets a nonexistent stage, and even when live pointed at a different simulator.

Toolchain reuse. The RV32IMA/ILP32 multilib in our installed `riscv64-unknown-elf-gcc` covers PicoRV32 with no new toolchain, confirmed by a trial assemble-and-link of the target diagnostic.

Results. The adapter extension (a new `Literal` entry, two `sims` flags, mirrored tests) landed with full unit coverage and no regressions to the `ariane` and `sparc` paths. The build takes 57 seconds, faster than Ariane’s since it skips the bootrom/device-tree chain, but the first run ended at `maxcycles`. The binary’s symbols and entry point matched the run’s trap addresses (`objdump -t/-f` against `symbol.tbl`), ruling out a build or toolchain mismatch, and

its transcript matched a passing Ariane fixture line for line until the fixture’s Hit Good trap, which never appeared: the core itself never reached its trap address.

Waveform tracing found the root cause: three RTL/testbench bugs, PicoRV32’s `resetn/booted` self-boot gate, the manycore monitor’s `active_thread` tracking for pico’s tile, and a silicon BIST self-clear race that discarded pico’s first memory writes. With all three fixed, PicoRV32 **passes** under Verilator (`Simulation -> PASS (HIT GOOD TRAP)`). The bugs surfaced only because MACE separates a successful build from a correct run instead of reporting a false pass.

Multi-tile pico. The fixes generalize: on 2x2 and 4x4 meshes, every tile reaches Hit Good trap on `addi.S` (`Simulation -> PASS`). `barrier_atomic.c` never runs on pico at any mesh size, for a simpler reason: pico’s OpenPiton integration has only an assembler, no C compiler or `crt.S/syscalls.c`, so the diag never compiles and every tile idles at its reset vector on an empty memory image. This is a toolchain gap, not an RTL bug, and not the workload/mesh mismatch an earlier, unverified triage diagnosis assumed. The atomic hardware that workload needs does work: the upstream `amoadd_w.S` test, pure assembly, passes on pico.

6. Beyond the Proposal: Interface, Coverage, Portability

The `mace` command starts an interactive shell modeled on Yosys and OpenROAD: `read_verilog`, `top_module`, `read_spec`, `set_core`, then `run`, giving MACE a standard EDA-style front end. If `top_module` names a core `chia_openpiton` does not support, `run` immediately flags the incompatibility instead of attempting a flawed integration.

Verilator code coverage (`run -coverage`) is a second verification axis beyond pass/fail. OpenPiton’s hand-written testbench never called Verilator’s coverage-write API, so a clean run produced no data until we patched it to call that API directly. A run (1x1 Ariane, `barrier_atomic.c`) reports 36.00% (8787/24311).

MACE’s loop also runs end to end, unmodified, against Google’s Vertex Gemini backend and OpenCode, confirmed with a full passing run.

7. Evaluation

We compare three approaches: (a) manual mesh scaling with a time log, (b) a single LLM prompt for a configuration applied once with no tools, verification, or retry, and (c) the full MACE loop.

Three cores, two baselines. We ran (b) and (c) on the same objective, verify `barrier_atomic.c` on a 1x1 mesh, for all three cores, with Gemini 2.5 Flash throughout (Figure 2, left).

(b) fails all three cores, each in a different way. On ariane, the one-shot config built but failed verification: a

wrong guess. On sparc, it built and reached a run attempt that failed silently, with no verdict at all. On pico, `PitonConfig`’s construction-time validation rejected the LLM’s config (`l15_size=0`) before any hardware ran; (b) has no retry path for a malformed response.

(c) passes ariane: 4/4 tasks, 741.7s, about 4x (b)’s cost. A second ariane gate workload, `producer_consumer.c`, also passed (2118.02s, 2 tasks), coverage beyond one program. On sparc and pico, (c) exhausts its retry budget on `barrier_atomic.c` instead of passing; MACE’s triage and post-mortem told the two failures apart, and we verified both independently.

sparc’s failure has two layers. The first is an ISA gap: `barrier_atomic.c` uses `util.h`’s `ATOMIC_FETCH_OP` macro, which is RISC-V inline assembly, and OpenSPARC T1 has no C diag environment, so no portable fix exists. Pure-assembly diagnostics, including OpenPiton’s CI-verified `princeton-test-test.s`, avoid that gap but expose a second: every sparc run reached `max_cycle` with no verdict, because `pc_cmp.v.pyv`’s `RTL_SPARC0` branch never assigned `active_thread`, unlike the Ariane and pico branches. Mirroring their pattern fixed the monitor, which now tracks all four threads and reports `FAIL (TIMEOUT)` instead of silence. The threads themselves still make no forward progress, even at 100x the default timeout; that remains open.

MACE’s triage blamed pico’s failure on a workload/mesh mismatch (a 1x1 mesh cannot supply `barrier_atomic.c`’s multi-participant barrier). That unverified LLM diagnosis was wrong: `barrier_atomic.c` never compiles for pico at any mesh size (Section 5). On `addi.S`, the loop passes pico in one task at every size, end to end: 1x1 in 115.5s, 2x2 in 70.7s, and 4x4 in 293.2s. That needed one new capability, a `CONFIG_RTL: planner` directive so a task can request `CONFIG_DISABLE_BIST_CLEAR` on top of the mesh’s defaults. The fix was named in the objective, so this shows the loop applying a known fix, not discovering one.

The loop passes mesh shapes where manual scaling stalled. Baseline (a), run before the fixes below, got stuck at 2x2 (build passes, run hangs) and never reached 4x4. Three latent bugs in OpenPiton’s shared code were responsible. A hardcoded trace-file name in `cva6.sv` made every tile write the same file, so tiles beyond 0 looked like they never booted. The upstream exit barrier in `syscalls.c`, used by every ariane diagnostic, polls a cross-tile atomic counter with a plain load, the same staleness bug `atomic_read()` already works around in our gate workloads. And the monitor’s `finish_mask` register was a fixed 32-bit Verilog integer under Verilator, capping verifiable coverage at 8 tiles: a 4x4 run reported `PASS` after only 8 of 16 tiles finished, as each tile’s execution trace showed. With all three fixed, the full loop passes both shapes with every tile confirmed (Figure 2, right): 2x2 in 750.1s across four tasks, including an L1D-associativity variant the planner chose to build and verify, and 4x4 in 250.9s across two tasks on a cached build.

Phase 2’s fan-out (two tasks, two checkouts, through

Core	(b) one-shot LLM	(c) full loop
ariane	fail, 186.5s	pass, 741.7s
sparc	fail, 766.0s	budget_exceeded, 966.3s
pico	fail (unparseable), 32.6s	budget_exceeded, 1250.5s

Figure 2. **Left:** all three cores, same 1x1 objective. (b) never passes. On sparc/pico, (c) exhausts its retry budget instead of reporting a false pass; only ariane **passes**, under (c). **Right:** manual mesh scaling (a) vs. (c), ariane only. Green=pass, red=hang, gray=not reached. The loop passes both shapes (a) got stuck on.

baseline (c)’s path) measured 30s parallel versus 57s serial, matching Phase 1’s result.

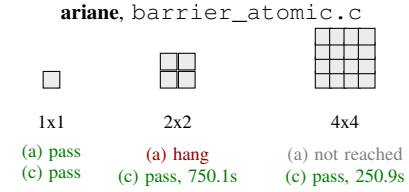
One validation limitation remains: the detect-failure, diagnose, replan, pass cycle is proven only at the mechanism level (a stateful stub that fails once, then passes). No passing run above needed it, not even a forced extreme first config (a 128-byte, direct-mapped L1, confirmed in the build) that we and the planner both expected to fail. The one real failure it met, a pico 4x4 build cached before the `finish_mask` fix, it did not recover from: a build ID covers configuration, not source edits, so all three replans reused the stale build while triage blamed the RTL. A clean rebuild passed.

8. Limitations and Open Issues

GCP dispatch. A task needing a tailnet-relayed GCP worker first never ran because CHIA sets three proxy environment variables during `chia up` that `chia job submit` inherits but our manually launched drivers did not; exporting them took the task from a six-minute hang to a two-second run. A later attempt on the same topology hung again with that fix in place, and relay logs plus a raylet `strace` show no outbound activity, pointing at Ray’s scheduling under a proxied topology rather than CHIA. That follow-up remains open.

4x4 memory pressure, two causes, both fixed. The upstream-validated 4x4 shape first ran out of memory (22.33GB of 23.47GB) under unconstrained parallel compilation, and limiting it through `MAKEFLAGS` did not help: OpenPiton’s Verilator build step passed a bare, unlimited `-j` that overrode it, confirmed through `/proc`. Removing that `-j` fixed it but exposed a race in the boot ROM’s Makefile, where one job’s cleanup deletes a device-tree blob another job still needs, so we build 4x4 with `MAKEFLAGS=-j1`.

Future work: generality over one-off adapters. All three cores here already had upstream OpenPiton/L15 adapters (SPARC’s CCX transducer, Ariane’s L15 adapter in CVA6, PicoRV32’s transducer); a core without one, such as Ibex or Rocket Chip, needs it written first. The existing three share a shape: a per-core RTL directory, a transducer between the core’s memory interface and L15, and a core-select arm in the tile template. The next step is turning that shape into a template and adding 2-4 more cores, with MACE’s loop verifying each iteration of the adapter RTL in Verilator while a hardware engineer writes it. A second



direction is a dashboard over the run database, showing dispatch, build, and verification live instead of through raw trace files.

9. Conclusion

MACE shows that an agentic loop can plan, build, and verify multicore hardware, gated by RTL simulation instead of an agent’s word. Along the way, the project fixed environment and RTL bugs and added a third core, and the loop passed 2x2 and 4x4 meshes of both Ariane and PicoRV32. Precision, not a single pass, is the point.

References

- [1] M. Chen *et al.*, “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.
- [2] M. Liu *et al.*, “VerilogEval: Evaluating large language models for Verilog code generation,” in *Proc. ICCAD*, 2023.
- [3] J. Balkind *et al.*, “OpenPiton: An open source manycore research framework,” in *Proc. ASPLOS*, 2016, pp. 217–232.
- [4] W. Snyder *et al.*, “Verilator,” <https://verilator.org>.
- [5] A. Cui *et al.*, “CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research,” arXiv:2606.27350, 2026.
- [6] P. Moritz *et al.*, “Ray: A distributed framework for emerging AI applications,” in *Proc. OSDI*, 2018, pp. 561–577.
- [7] J. Balkind *et al.*, “BYOC: A “Bring Your Own Core” framework for heterogeneous-ISA research,” in *Proc. ASPLOS*, 2020, pp. 699–714.
- [8] F. Zaruba and L. Benini, “The cost of application-class processing: Energy and performance analysis of a Linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology,” *IEEE Trans. VLSI Syst.*, vol. 27, no. 11, pp. 2629–2640, 2019.
- [9] C. X. Wolf, “PicoRV32 – a size-optimized RISC-V CPU,” <https://github.com/YosysHQ/picorv32>.
- [10] M. Liu *et al.*, “ChipNeMo: Domain-adapted LLMs for chip design,” arXiv:2311.00176, 2023.
- [11] J. Blocklove *et al.*, “Automatically improving LLM-based Verilog generation using EDA tool feedback,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 30, no. 6, 2025.
- [12] Y. Tsai, M. Liu, and H. Ren, “RTLFixer: Automatically fixing RTL syntax errors with large language model,” in *Proc. DAC*, 2024.
- [13] C.-T. Ho, H. Ren, and B. Khailany, “VerilogCoder: Autonomous Verilog coding agents with graph-based planning and abstract syntax tree (AST)-based waveform tracing tool,” in *Proc. AAAI*, 2025.
- [14] Y. Zhao *et al.*, “MAGE: A multi-agent engine for automated RTL code generation,” in *Proc. DAC*, 2025.
- [15] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. ICLR*, 2024.
- [16] E. Dolstra, M. de Jonge, and E. Visser, “Nix: A safe and policy-free system for software deployment,” in *Proc. LISA*, 2004, pp. 79–92.